

SIH25033 · PROJECT REPORT · 20 AUGUST 2026

InternSetu

Internship matching, allocation, resume parsing, feedback, and analytics platform.

InternSetu connects candidates with government-managed internship opportunities through an explainable weighted matching engine and a role-based operational dashboard.

Technology: React 19 · Vite · Node.js · Express · MongoDB · Mongoose · FastAPI · pdfminer.six · JWT

1. Executive Summary

InternSetu is a three-part system: a React/Vite web client, an Express/MongoDB application backend, and a separate FastAPI resume parser. Candidates can register, maintain profiles, upload PDF resumes, browse internships, receive recommendations, view allocations, and submit feedback. Government administrators and officers can create internships, inspect candidates, run allocation, review feedback, and view analytics.

The matching engine scores skills, eligibility, role, location, sector, and experience. Eligibility is a hard gate. Eligible candidates are ranked by weighted score, assigned to available seats, and then placed on a waitlist when seats are exhausted.

2. Problem, Objectives, and Scope

Problem

Manual internship matching is slow, difficult to audit, and inconsistent as candidate and opportunity volume grows.

Objectives

- Centralize candidate profiles and internship opportunities.
- Extract useful information from PDF resumes.
- Provide explainable candidate-to-internship scores.
- Allocate seats fairly using ranked results and waitlists.
- Provide feedback and analytics for program monitoring.
- Separate candidate and government workflows with role-based authorization.

Delivered scope

Authentication, profiles, PDF upload and parsing, internship catalogue, recommendations, allocation, waitlists, allocations history, feedback, feedback statistics, analytics, upload validation, and local development configuration are implemented.

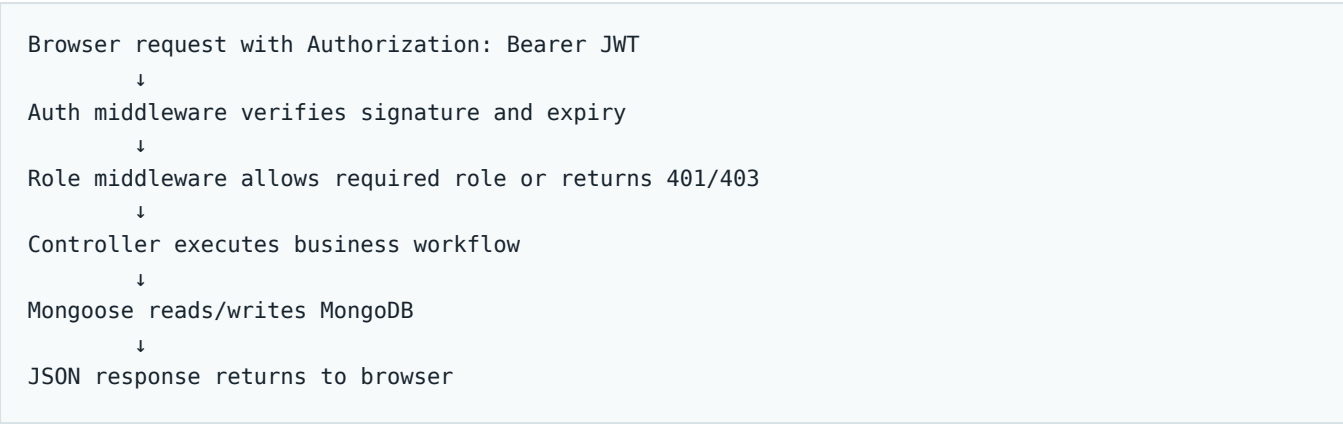
Current boundaries: parsing is heuristic and vocabulary-based; matching uses normalized exact comparisons for several dimensions; frontend navigation is React state rather than browser routing; formal backend/frontend test suites are not yet present.

3. High-Level Architecture



Component	Technology	Responsibility	Address
Frontend	React 19, Vite	UI, session state, API requests, role views	localhost:5173
Backend	Node.js, Express 5	REST API, auth, authorization, workflows	localhost:5000
Database	MongoDB/Mongoose	Persistent application data	127.0.0.1:27017
Parser	FastAPI, Uvicorn, pdfminer.six	PDF text extraction and field detection	127.0.0.1:8000

Request security flow

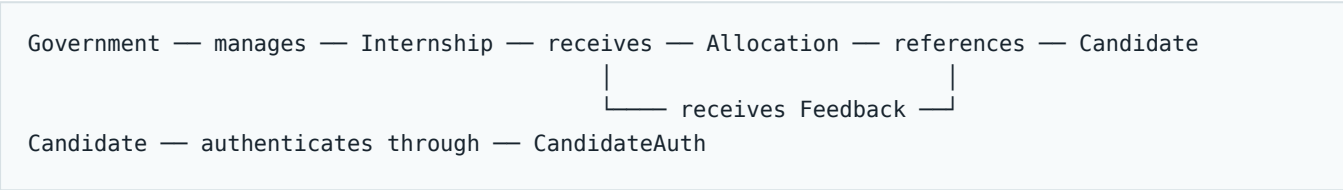


4. Backend Architecture

Backend/server.js loads environment variables, connects to MongoDB, and listens on port 5000.
Backend/src/app.js enables CORS and JSON parsing, mounts route groups, and exposes GET / as the health endpoint.

Route group	Main responsibilities	Access
/api/candidates	Register/login, profile, resume, recommendations, allocations	Candidate or government as appropriate
/api/auth	Government registration and login	Public auth endpoints
/api/internships	Create, list, retrieve internships	Create: admin/officer; read: public
/api/allocation	Run matching and retrieve results	Government role
/api/feedback	Submit feedback, list feedback, statistics	Candidate or government role
/api/analytics	Platform overview metrics	Admin/officer

Data model



Model	Key information
Candidate	Profile, education, skills, preferences, experience, eligibility, resume URL
CandidateAuth	Email/password record linked to Candidate
Government	Identity, department, hashed password, admin/officer role
Internship	Organization, role, sector, skills, eligibility criteria, seats, deadline, status
Allocation	Candidate/internship references, score, breakdown, ALLOCATED/WAITLIST status
Feedback	Candidate/internship/allocation references, rating 1-5, comment; unique candidate/internship rule

5. Resume Parsing Workflow

```

Select PDF → Validate PDF and 5 MB limit → Multer stores file
  → Backend sends multipart request to FastAPI /parse-resume
  → pdfminer extracts text
  → Detect known skills, education lines, experience lines
  → Return JSON → Update Candidate document → Refresh profile

```

The parser accepts PDFs only. It detects skills from a fixed vocabulary, education from degree keywords, and experience from lines following experience headings, with a simple date-range fallback. The backend uses `PARSER_BASE_URL` and Axios to call the parser.

6. Matching and Allocation

Component	Weight	Behavior
Skills	40%	Matched required skills divided by required skills
Eligibility	20%	Hard gate for eligibility, degree, branch, graduation year
Role	15%	Predefined frontend, backend, software, full-stack, data, ML groups
Location	10%	Normalized exact match
Sector	10%	Normalized exact match
Experience	5%	Full score when met; proportional partial score otherwise

```

score = 0.40 skills + 0.20 eligibility + 0.15 role
       + 0.10 location + 0.10 sector + 0.05 experience

```

```
Government selects internship
  ↓ POST /api/allocation/run/:internshipId
Load internship and candidates → exclude candidates allocated elsewhere
  ↓
Apply eligibility gate → calculate score and breakdown → sort descending
  ↓
Ranks within available seats: ALLOCATED
Remaining eligible ranks: WAITLIST
  ↓
Persist results and return ranked allocation response
```

Prior results for the selected internship are deleted before a new run. Recommendations reuse scoring but are calculated on demand and are not persisted. Allocation results preserve component breakdowns for explainability.

7. Frontend and User Workflows

Candidate journey

```
Register/sign in → Complete profile → Upload resume → Parser enriches profile
  → Browse internships → View recommendations → Check allocations → Submit feedback
```

Government journey

```
Register/sign in → Create internship → Review candidates → Run allocation
  → Inspect allocated/waitlisted results → Review feedback → View analytics
```

`App.jsx` selects authentication, candidate, or government views. `AuthContext.jsx` persists sessions in local storage under `sih25033-session`. The active API client is `api2.js`, configured with `VITE_API_BASE_URL`, and attaches bearer tokens automatically.

8. Deployment and Startup

Process	Command
Backend	<code>cd Backend ; npm run dev</code>
Parser	<code>.\.venv\Scripts\Activate.ps1 ; python -m uvicorn Parser.app:app --host 127.0.0.1 -port 8000</code>
Frontend	<code>cd Frontend ; npm run dev</code>

```
Backend/.env
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/sih25033
JWT_SECRET=replace_with_secure_secret
PARSER_BASE_URL=http://localhost:8000

Frontend/.env
VITE_API_BASE_URL=http://localhost:5000/api
```

Operational note: all three development terminals must stay open. If the backend stops, the browser reports Failed to fetch because port 5000 is unavailable.

9. Test Strategy and Cases

The repository has operational/manual parser and API scripts, but no formal backend or frontend test suite. The backend test script is a placeholder and the parser test files make live HTTP requests without assertions or a test runner. The following acceptance cases define the recommended coverage.

ID	Scenario	Expected result
ENV-01	MongoDB service and port check	Running service and port 27017 available
ENV-02	Backend startup	Port 5000 and Database Connected
ENV-03	GET /	Health JSON returned
ENV-04	Parser startup and OpenAPI request	Startup complete and HTTP 200
AUTH-01	Valid candidate registration/login	Profile and JWT returned
AUTH-02	Missing token or wrong role	401 or 403 respectively
PARSE-01	Valid text PDF upload	Skills, education, experience JSON and profile update
PARSE-02	Non-PDF or over-5 MB upload	Controlled validation error
MATCH-01	All dimensions match	Score 100 with breakdown
MATCH-02	Eligibility fails	Not allocated regardless of other score
MATCH-03	Seats fewer than candidates	Top ranks allocated, remainder waitlisted
MATCH-04	Repeat allocation run	Old results replaced, no duplicates
FB-01	Candidate submits feedback	Rating/comment stored
FB-02	Duplicate candidate/internship feedback	Uniqueness rule prevents duplicate
AN-01	Officer requests analytics	Totals, unallocated count, feedback count, average rating
UI-01	Candidate session and refresh	Dashboard opens and session persists
UI-02	Government allocation screen	Scores, breakdown, and statuses render

10. Verification Performed

Check	Observed result
MongoDB service	Running, automatic startup
MongoDB port	TcpTestSucceeded: True
Backend	Port 5000; Database Connected
Backend health	{"message": "Internship Matcher API is running"}
Parser	Uvicorn running on port 8000
Parser OpenAPI	HTTP 200 OK
Frontend	Vite application served on port 5173

These are smoke and environment checks, not a substitute for a formal automated regression suite.

11. Risks and Next Improvements

- Improve parser accuracy for synonyms, scanned PDFs, and unusual layouts.
- Use semantic skill matching and location normalization while preserving score explainability.
- Add Jest/Supertest backend tests and Vitest/React Testing Library frontend tests.
- Create parser fixtures with expected output and negative cases.
- Move resume files to object storage for production.
- Add request schemas, pagination, audit logs, concurrency control, and production security settings.

12. Conclusion: What We Achieved

This project achieved a working end-to-end internship matching foundation, not merely a static interface. Candidates can enter the platform, build profiles, enrich them with resume parsing, discover opportunities, receive ranked recommendations, see allocation outcomes, and provide feedback. Government users can manage opportunities, inspect candidates, run transparent allocation, and monitor outcomes through feedback and analytics.

The project also established clear service boundaries between browser, Express API, MongoDB, and Python parsing service. Explicit weighted scoring and stored score breakdowns make allocation results understandable and reviewable. The system is ready as a local demonstration and a strong base for production hardening; automated testing and operational improvements are the next maturity step.